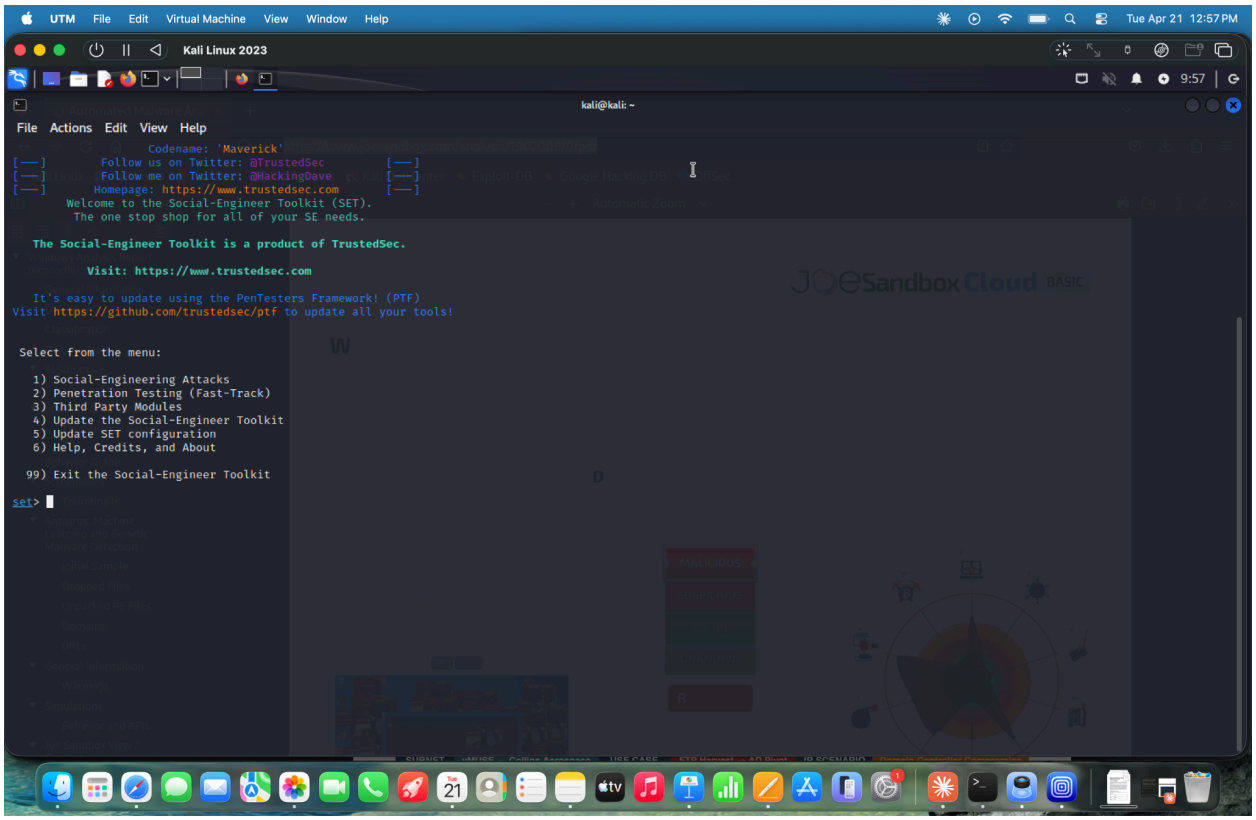
This report outlines the simulation of a **Credential Harvester Attack** using the Social-Engineer Toolkit (SET). It details the methodology, the mechanics of the capture, and the resulting data found in your screenshots.



Technical Analysis Report: Credential Harvester Attack Simulation

1. Executive Summary

The objective of this task was to demonstrate the effectiveness of a website cloning attack to capture user credentials. By leveraging the **Social-Engineer Toolkit (SET)**, a replica of a known login portal (Google) was hosted on a local attacker instance. The simulation successfully captured plaintext credentials and session parameters when a user interacted with the cloned site.

2. Methodology & Attack Vector

The attack followed the **Social-Engineering Attack Vector** path within SET:

1. **Select Attack Vector:** Website Attack Vectors (Option 2).

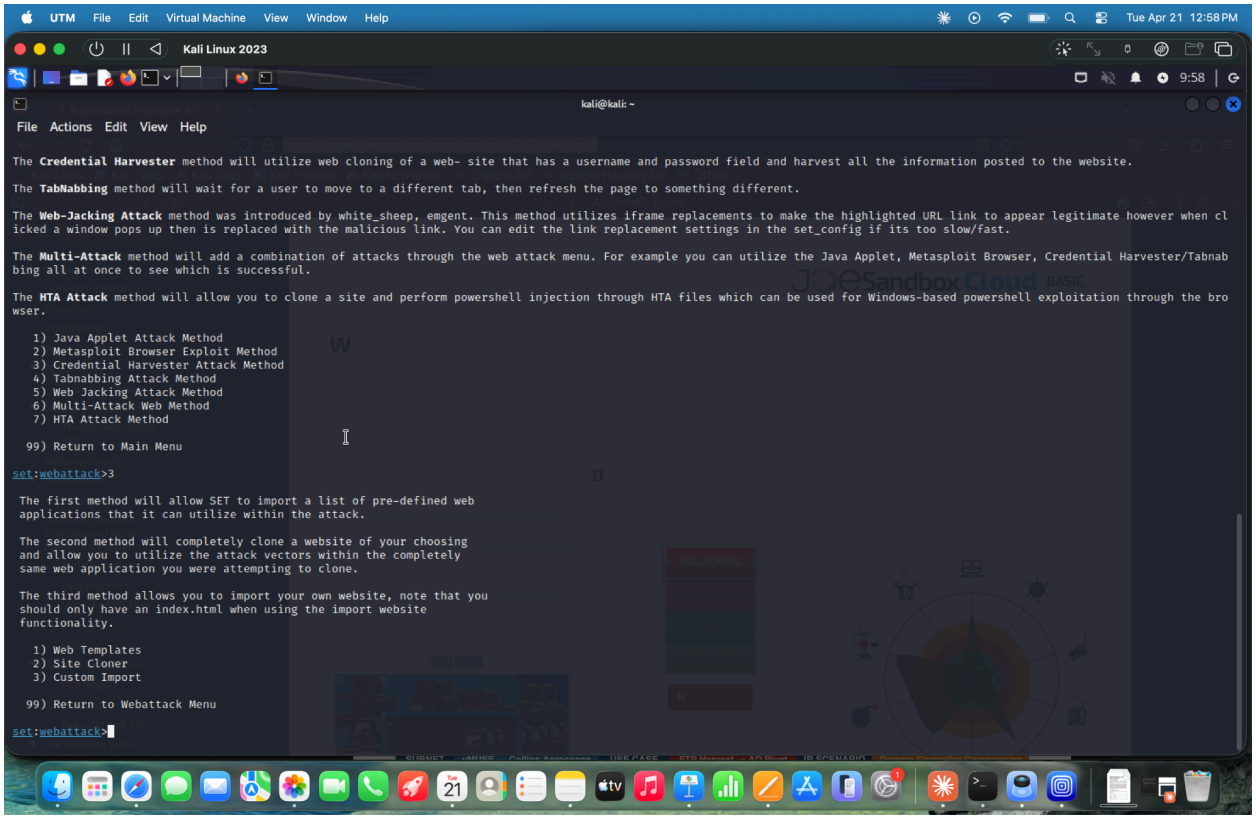
2. **Attack Method:** Credential Harvester Attack Method (Option 3).
3. **Deployment:** Site Cloner / Template (Option 2).
4. **Target:** Cloning `http://www.google.com` to intercept POST requests intended for the legitimate service.

---

3. Technical Process Breakdown

A. Environment Setup

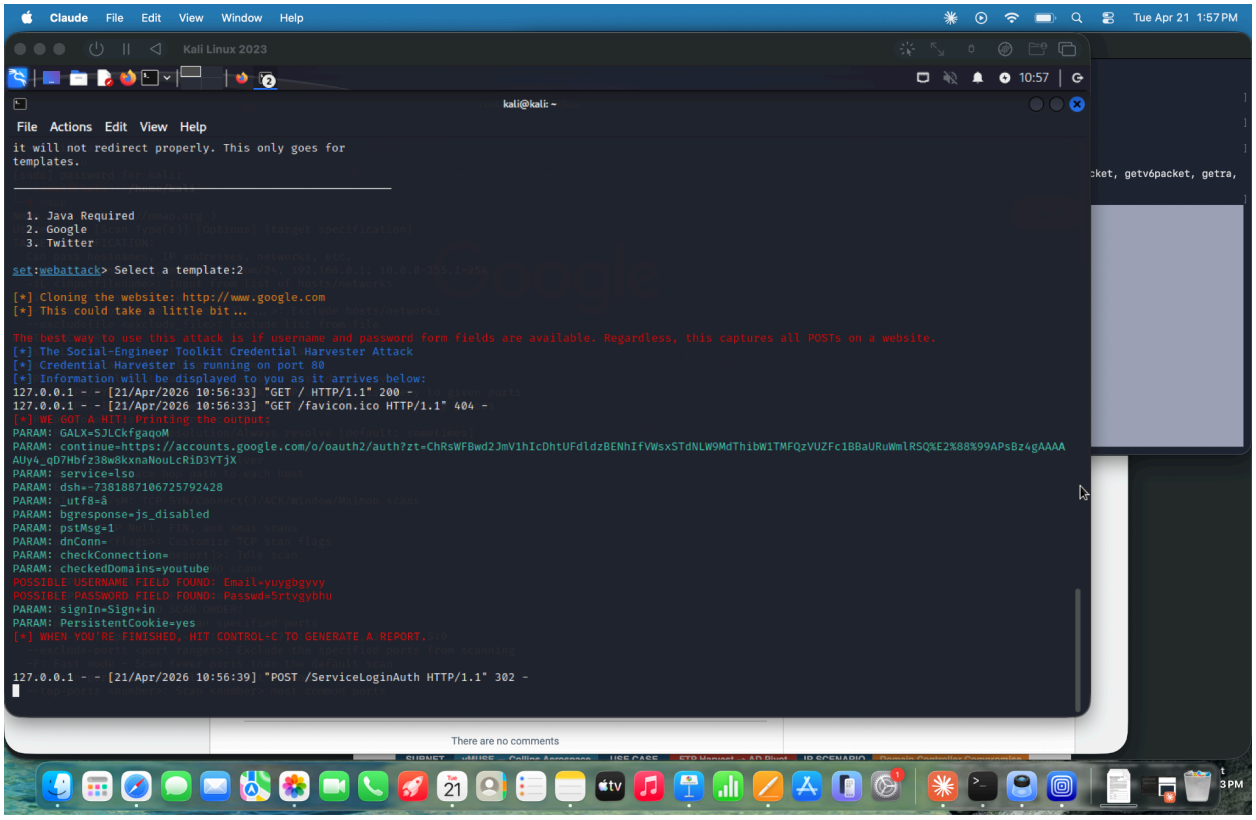
As seen in the screenshots, the attack was executed on a **Kali Linux** instance. SET initialized a "Credential Harvester" listening on **Port 80**.



B. The Cloning Process

SET performed a real-time "clone" of the target website's front-end code. This creates a visually identical login page. The toolkit then injects a script that redirects the `POST` action (the submission of the username and password) to the attacker's machine instead of the target's actual authentication server.

C. Data Interception



Once the victim enters their information, the following events occur:

- **HTTP GET Request:** The attacker logs show the victim accessing the root directory (`/`) and fetching the `favicon.ico`.
- **HTTP POST Request:** When the "Sign In" button is clicked, a `POST` request is sent to `/ServiceLoginAuth`. This is the moment the data is intercepted.

---

4. Results & Analysis of Captured Data

The second and third screenshots show a successful "HIT." The toolkit automatically parsed the following critical information from the intercepted POST request:

Field Type	Captured Value	Significance
POSSIBLE_USERNAME	`yuygbgyvy`	The string entered into the "Email" field.
POSSIBLE_PASSWORD	`5rtvgybhu`	The plaintext string entered into the "Password" field.

PARAM: continue	`https://accounts.google.com/...`	The legitimate URL the user was supposed to be redirected to.
HTTP Status	`302`	Indicates a redirect occurred after the POST, often used to send the user back to the real site to hide the attack.

Technical Observation:

The capture includes complex URL parameters (e.g., `GALX`, `service=lso`, `dsh`). These are part of the OAuth2 and Google authentication flow. Capturing these can sometimes allow an attacker to perform "Session Hijacking" or "Replay Attacks" if the tokens are not properly scoped or if the session remains active.

---

5. Defensive Recommendations

To mitigate the risks identified in this simulation, the following controls should be implemented:

- **Multi-Factor Authentication (MFA):** Even with captured plaintext passwords, MFA (TOTP or Hardware Keys) would prevent the attacker from gaining access.
- **Domain Monitoring:** Organizations should monitor for "typosquatting" or look-alike domains that could be used to host these SET clones.
- **Email Filtering:** Implementation of SPF, DKIM, and DMARC to prevent phishing emails from reaching the end-user's inbox.
- **User Awareness Training:** Teaching users to verify the URL and SSL certificate (HTTPS) before entering credentials.